机器视觉实验报告

实验一　光源及图像采集、系统标定

```
* 生成 7×7 标定板描述文件和 PS 文件，用于后续相机标定
gen_caltab (7, 7, 0.01, 0.5, 'E:/BCC/MVExp1/caltab.descr',
'E:/BCC/MVExp1/caltab.ps')
* Calibration 01: Code generated by Calibration 01
* 保存标定得到的相机内参，包括镜头模型、焦距/畸变、像元尺寸、主点和图像尺寸
CameraParameters := ['area_scan_division',0.0436375,-602.462,5.30232e-
06,5.3e-06,656.827,490,1280,1024]
* 保存标定得到的相机外参位姿参数
CameraPose := [0.0018186,-0.000894141,0.929627,359.057,359.711,359.324,0]
* 标定参数部分暂停，便于检查生成的参数
stop ()
* Image Acquisition 02: Code generated by Image Acquisition 02
* 打开 GigE Vision 工业相机，设备名对应实验平台上的 Basler 相机
open_framegrabber ('GigEVision2', 0, 0, 0, 0, 0, 0, 'progressive', -1,
'default', -1, 'false', 'default', '0030532476d8_Basler_acA130060gm', 0,
-1, AcqHandle)
* 启动图像采集流程
grab_image_start (AcqHandle, -1)
* 打开 HALCON 图形显示窗口
dev_open_window(0, 0, 512, 512, 'black', WindowHandle)
* 进入循环，持续从相机采集并显示图像
while (true)
* 异步抓取一帧图像
    grab_image_async (Image, AcqHandle, -1)
    * Image Acquisition 02: Do something
* 将当前采集到的图像显示在窗口中
    dev_display(Image)
endwhile
* 关闭相机采集句柄，释放设备资源
close_framegrabber (AcqHandle)
```

实验二　物体的定位实验

```
*
* Matching 01: ************************************************
* Matching 01: BEGIN of generated code for model initialization
* Matching 01: ************************************************
* 设置形状模板边界处理方式，避免模板边界外区域影响匹配
set_system ('border_shape_models', 'false')
*
```

```
* Matching 01: Initialize acquisition
* 打开工业相机，为后续在线采图和实时匹配做准备
open_framegrabber ('GigEVision2', 0, 0, 0, 0, 0, 0, 'progressive', -1,
'default', -1, 'false', 'default', '0030532476d8_Basler_acA130060gm', 0,
-1, AcqHandle)
*
* Matching 01: Obtain the model image
* 读取参考图像，用该图像中的工件区域创建模板
read_image (Image, 'E:/BCC/MVExp2/Image__2026-04-29__22-53-46.bmp')
*
* Matching 01: Build the ROI from basic regions
* 生成矩形 ROI，框选工件模板区域
gen_rectangle1 (ModelRegion, 262.711, 233.866, 742.261, 965.81)
*
* Matching 01: Reduce the model template
* 将图像处理范围限制到 ROI，得到模板图像
reduce_domain (Image, ModelRegion, TemplateImage)
*
* Matching 01: Create the shape model
* 创建基于形状的模板模型，设置旋转搜索范围和金字塔层数等参数
create_shape_model (TemplateImage, 7, rad(0), rad(360), rad(0.2735),
['point_reduction_high','no_pregeneration'], 'use_polarity', [23,30,72],
3, ModelID)
*
* Matching 01: Get the model contour for transforming it later into the
image
* 提取模板轮廓，便于在匹配位置显示模板外形
get_shape_model_contours (ModelContours, ModelID, 1)
*
* Matching 01: Get the reference position
* 计算模板 ROI 的中心位置，作为模板参考点
area_center (ModelRegion, ModelRegionArea, RefRow, RefColumn)
* 建立从模板局部坐标到参考位置的刚性变换
vector_angle_to_rigid (0, 0, 0, RefRow, RefColumn, 0, HomMat2D)
* 根据刚性变换把模板轮廓移动到参考位置
affine_trans_contour_xld (ModelContours, TransContours, HomMat2D)
*
* Matching 01: Display the model contours
dev_display (Image)
dev_set_color ('green')
dev_set_draw ('margin')
* 显示模板 ROI
dev_display (ModelRegion)
* 显示模板轮廓
dev_display (TransContours)
stop ()
*
* Matching 01: END of generated code for model initialization
```

```
* Matching 01:  * * * * * * * * * * * * * * * * * * * * * * * * *
* Matching 01: BEGIN of generated code for model application
*

    dev_close_window()
* 按图像尺寸打开显示窗口
    dev_open_window_fit_image(Image, 0, 0, -1, -1, WindowHandle)

while (true)
    *
    * Matching 01: Obtain the image
* 从相机抓取待匹配图像
    grab_image (Image, AcqHandle)
    *
    * Matching 01: Find the model
* 在当前图像中搜索形状模板，输出匹配位置、角度和分数
    find_shape_model (Image, ModelID, rad(0), rad(360), 0.5, 0, 0.5,
'least_squares', [7,1], 0.7, Row, Column, Angle, Score)
    *
    * Matching 01: Transform the model contours into the detected
positions
    dev_display (Image)
* 遍历所有匹配结果，并逐个显示
    for I := 0 to |Score| - 1 by 1
* 初始化二维齐次变换矩阵
        hom_mat2d_identity (HomMat2D)
* 根据匹配角度旋转模板轮廓
        hom_mat2d_rotate (HomMat2D, Angle[I], 0, 0, HomMat2D)
* 根据匹配行列坐标平移模板轮廓
        hom_mat2d_translate (HomMat2D, Row[I], Column[I], HomMat2D)
        affine_trans_contour_xld (ModelContours, TransContours, HomMat2D)
        dev_set_color ('green')
        dev_display (TransContours)
* 将弧度角转换为角度值，便于显示
        AngleDeg := deg(Angle[I])
* 组织匹配结果文本，包括 Row、Column、Angle 和 Score
        Message := 'Row: ' + Row[I] + '\nColumn: ' + Column[I] +
'\nAngle: ' + AngleDeg + '\nScore: ' + Score[I]
* 在窗口中显示匹配结果信息
        disp_message (WindowHandle, Message, 'window', 30, 30, 'black',
'true')
        * stop ()
    endfor
endwhile
*
* Matching 01: ****************************************
* Matching 01: END of generated code for model application
* Matching 01: ****************************************
```

```
*
```

实验三　1D/2D 测量实验

1D 测量

```
* Measure 01: Code generated by Measure 01
* Measure 01: Prepare measurement
* 设置边缘幅值阈值，低于该幅值的边缘不作为有效测量边缘
AmplitudeThreshold := 66
* 设置测量矩形半宽，即测量 ROI 垂直于测量线方向的宽度
RoiWidthLen2 := 15
* 开启整型缩放显示设置
set_system ('int_zooming', 'true')
* Measure 01: Coordinates for line Measure 01 [0]
* 设置测量线起点行坐标
LineRowStart_Measure_01_0 := 357.982
* 设置测量线起点列坐标
LineColumnStart_Measure_01_0 := 415.762
* 设置测量线终点行坐标
LineRowEnd_Measure_01_0 := 359.768
* 设置测量线终点列坐标
LineColumnEnd_Measure_01_0 := 513.659
* Measure 01: Convert coordinates to rectangle2 type
* 计算矩形测量 ROI 的中心行坐标
TmpCtrl_Row := 0.5*(LineRowStart_Measure_01_0+LineRowEnd_Measure_01_0)
* 计算矩形测量 ROI 的中心列坐标
TmpCtrl_Column := 0.5*
(LineColumnStart_Measure_01_0+LineColumnEnd_Measure_01_0)
* 计算测量线在行方向上的差值
TmpCtrl_Dr := LineRowStart_Measure_01_0-LineRowEnd_Measure_01_0
* 计算测量线在列方向上的差值
TmpCtrl_Dc := LineColumnEnd_Measure_01_0-LineColumnStart_Measure_01_0
* 计算测量矩形方向角
TmpCtrl_Phi := atan2(TmpCtrl_Dr, TmpCtrl_Dc)
* 计算测量矩形长度方向的半长
TmpCtrl_Len1 := 0.5*sqrt(TmpCtrl_Dr*TmpCtrl_Dr + TmpCtrl_Dc*TmpCtrl_Dc)
* 设置测量矩形宽度方向的半宽
TmpCtrl_Len2 := RoiWidthLen2
* Measure 01: Create measure for line Measure 01 [0]
* Measure 01: Attention: This assumes all images have the same size!
* 根据中心、角度、半长和半宽生成一维测量句柄
gen_measure_rectangle2 (TmpCtrl_Row, TmpCtrl_Column, TmpCtrl_Phi,
TmpCtrl_Len1, TmpCtrl_Len2, 750, 600, 'nearest_neighbor',
MsrHandle_Measure_01_0)
* Measure 01:
************************************************************
```

```
* Measure 01: * The code which follows is to be executed once /
measurement *
* Measure 01:
************************************************************
* Measure 01: Load image
* 读取待测工件图像
read_image (Image, 'E:/BCC/MVExp3/imagetomearsure.bmp')
* Measure 01: Execute measurements
* 执行一维边缘对测量，得到两条边缘的位置、幅值、宽度和间距
measure_pairs (Image, MsrHandle_Measure_01_0, 7.4, AmplitudeThreshold,
'all', 'all', Row1_Measure_01_0, Column1_Measure_01_0,
Amplitude1_Measure_01_0, Row2_Measure_01_0, Column2_Measure_01_0,
Amplitude2_Measure_01_0, Width_Measure_01_0, Distance_Measure_01_0)
* Measure 01: Do something with the results
dev_close_window()
* 打开适配图像尺寸的显示窗口
dev_open_window_fit_image (Image, 0, 0, -1, -1, WindowHandle)
dev_display(Image)
dev_set_draw('margin')
dev_set_color('green')
* 生成矩形ROI
* 生成测量 ROI 的显示区域
gen_rectangle2 (ROI, TmpCtrl_Row, TmpCtrl_Column, TmpCtrl_Phi,
TmpCtrl_Len1, TmpCtrl_Len2)
dev_display(ROI)
* 根据提取的边缘点生成XLD轮廓显示
* 根据第一条测量边缘点生成 XLD 轮廓，用于显示第一边缘
gen_contour_polygon_xld (EdgeFirst, \
                         [-
sin(TmpCtrl_Phi+rad(90))*TmpCtrl_Len2+Row1_Measure_01_0[0], \
                          -sin(TmpCtrl_Phi-
rad(90))*TmpCtrl_Len2+Row1_Measure_01_0[0]], \

[cos(TmpCtrl_Phi+rad(90))*TmpCtrl_Len2+Column1_Measure_01_0[0], \
                          cos(TmpCtrl_Phi-
rad(90))*TmpCtrl_Len2+Column1_Measure_01_0[0]])
* 根据第二条测量边缘点生成 XLD 轮廓，用于显示第二边缘
gen_contour_polygon_xld (EdgeSecond, \
                         [-
sin(TmpCtrl_Phi+rad(90))*TmpCtrl_Len2+Row2_Measure_01_0[0], \
                          -sin(TmpCtrl_Phi-
rad(90))*TmpCtrl_Len2+Row2_Measure_01_0[0]], \

[cos(TmpCtrl_Phi+rad(90))*TmpCtrl_Len2+Column2_Measure_01_0[0], \
                          cos(TmpCtrl_Phi-
rad(90))*TmpCtrl_Len2+Column2_Measure_01_0[0]])
dev_set_color('blue')
dev_display(EdgeFirst)
```

```
dev_set_color ('magenta')
dev_display (EdgeSecond)
* 组织宽度测量结果文本
Message := '宽: '+Width_Measure_01_0
* 在图像窗口中显示测量宽度
disp_message (WindowHandle, Message, 'window', 20, 20, 'black', 'true')
```

2D 测量

```
* 显示窗口
* 关闭已有显示窗口，避免窗口重叠
dev_close_window()
* 读取待测工件图像
read_image (Image, 'E:/BCC/MVExp3/imagetomearsure')
* 打开适配图像尺寸的显示窗口
dev_open_window_fit_image (Image, 0, 0, -1, -1, WindowHandle)
dev_display (Image)
* 单通道
* 将三通道图像转换为灰度图，便于后续边缘提取
rgb3_to_gray (Image, Image, Image, ImageGray)
* 创建轮廓
* 使用 Canny 算子提取亚像素边缘轮廓
edges_sub_pix (ImageGray, Edges, 'canny', 1, 20, 40)
* 选择圆形
* 根据圆度筛选小圆轮廓
select_shape_xld (Edges, SelectedXLD, 'circularity', 'and', 0.86514, 1)
* 根据圆度筛选大圆轮廓
select_shape_xld (Edges, SelectedXLD1, 'circularity', 'and', 0.68532,
0.86514)
* 拟合轮廓为圆，获取圆轮廓
* 对小圆轮廓进行圆拟合，得到圆心和半径
fit_circle_contour_xld (SelectedXLD, 'algebraic', -1, 0, 0, 3, 2, Row,
Column, Radius, StartPhi, EndPhi, PointOrder)
* 根据拟合结果生成小圆显示区域
gen_circle (Circle, Row, Column, Radius)
* 对大圆轮廓进行圆拟合，得到圆心和半径
fit_circle_contour_xld (SelectedXLD1, 'algebraic', -1, 0, 0, 3, 2, Row1,
Column1, Radius1, StartPhi1, EndPhi1, PointOrder1)
* 根据拟合结果生成大圆显示区域
gen_circle (Circle1, Row1, Column1, Radius1)
* 计算圆心距离
* 计算小圆圆心与大圆圆心之间的距离
distance_pp (Row, Column, Row1, Column1, Distance)
* 分割轮廓
* 将边缘轮廓分割为直线和圆弧等基本轮廓
segment_contours_xld (Edges, ContoursSplit, 'lines_circles', 5, 4, 2)
* 根据宽度特征选择轮廓
```

```
* 根据宽度特征筛选底部直线轮廓
select_shape_xld (ContoursSplit, SelectedXLD2, 'width', 'and', 302.49,
500)
* 拟合直线
* 对筛选出的直线轮廓进行直线拟合
fit_line_contour_xld (SelectedXLD2, 'tukey', -1, 0, 5, 2, RowBegin,
ColBegin, RowEnd, ColEnd, Nr, Nc, Dist)
* 根据拟合直线端点生成直线轮廓用于显示
gen_contour_polygon_xld (Line, [RowBegin, RowEnd], [ColBegin, ColEnd])
* 计算距离
* 计算大圆圆心到拟合直线的距离
distance_pl (Row1, Column1, RowBegin, ColBegin, RowEnd, ColEnd,
Distance1)

* 显示设置
dev_set_color ('red')
dev_set_line_width (3)
dev_set_draw ('margin')
* 显示结果
dev_display(Image)
dev_display(Circle)
dev_display (Circle1)
dev_display (Line)
* 显示大圆到直线的距离
disp_message (WindowHandle, '大圆到直线距离: '+Distance1, 'window', 40,
20, 'black', 'true')
* 绘制大小圆心之间的连线
disp_line (WindowHandle, Row1, Column1, Row, Column)
* 显示大小圆心距离
disp_message (WindowHandle, '大小圆心距离: '+Distance, 'window', 20, 20,
'black', 'true')
```

实验四　光学字符识别（**OCR**）实验

Blob 分析 + **MLP** 分类器 **OCR**

```
* 关闭已有显示窗口
dev_close_window ()
* 读取车牌图像
read_image (Image, 'E:/BCC/MVExp4/image.tif')
* 打开适配图像尺寸的显示窗口
dev_open_window_fit_image (Image, 0, 0, -1, -1, WindowHandle)
dev_display (Image)
* 将输入图像转换为灰度图
rgb1_to_gray (Image, GrayImage)
* 获取ROI
* 生成车牌字符区域 ROI
```

```
gen_rectangle1 (ROI_0, 231, 322, 301, 547)
* 图片剪裁
* 将处理范围限制到字符 ROI
reduce_domain (GrayImage, ROI_0, ImageReduced)
* 阈值分割
* 按灰度阈值分割字符区域
threshold (ImageReduced, Region, 128, 255)
* 断开连通域
* 将阈值分割区域拆分为独立连通域
connection (Region, ConnectedRegions)
* 特征选择
* 根据面积特征筛选有效字符区域
select_shape (ConnectedRegions, SelectedRegions, 'area', 'and', 100,
1000)
* 连通域排序
* 按照列方向对字符区域排序，保证识别结果顺序正确
sort_region (SelectedRegions, SortedRegions, 'first_point', 'true',
'column')
* 读分类器
* 读取 HALCON 内置 MLP OCR 分类器
read_ocr_class_mlp ('Industrial_0-9A-Z_NoRej.omc', OCRHandle)
* 取反图像
* 对字符 ROI 图像取反，使其符合分类器训练时的字符极性
invert_image (ImageReduced, ImageInvert)
* 字符识别
* 对排序后的多个字符区域执行 OCR 识别
do_ocr_multi_class_mlp (SortedRegions, ImageInvert, OCRHandle, Class,
Confidence)
* 显示
dev_display (Image)
* 统计筛选出的字符区域数量
count_obj (SelectedRegions, Number)
* 循环显示每一个字符的识别结果
for index:=0 to Number-1 by 1
* 在窗口上显示当前字符识别结果
    disp_message (WindowHandle, Class[index], 'window', 12, 12+index*15,
'black', 'true')
endfor
```

模板匹配 + **MLP** 分类器 **OCR**

```
* 关闭已有显示窗口
dev_close_window ()
* 读取车牌图像
* 读取参考图像，用汉字区域创建 NCC 模板
read_image (Image, 'E:/BCC/MVExp4/01.png')
```

```
* 打开适配图像尺寸的显示窗口
dev_open_window_fit_image (Image, 0, 0, -1, -1, WindowHandle)
dev_display (Image)
* 将车牌图像转换为灰度图
rgb1_to_gray (Image, GrayImage)
* 获取ROI
* 生成英文和数字字符区域 ROI
gen_rectangle1 (ROI_0, 394.593, 548.193, 508.94, 932.193)
* 图片剪裁
* 将处理范围限制到字符 ROI
reduce_domain (GrayImage, ROI_0, ImageReduced)
* 阈值分割
* 按灰度阈值提取字符区域
threshold (ImageReduced, Region, 128, 255)
* 断开连通域
* 将字符区域拆分为独立连通域
connection (Region, ConnectedRegions)
* 特征选择
* 根据面积筛选有效字符连通域
select_shape (ConnectedRegions, SelectedRegions, 'area', 'and', 600,
3000)
* 连通域排序
* 按列方向对字符区域排序，保证识别顺序
sort_region (SelectedRegions, SortedRegions, 'first_point', 'true',
'column')
* 读分类器
* 读取 HALCON 内置 MLP OCR 分类器
read_ocr_class_mlp('Industrial_0-9A-Z_NoRej.omc', OCRHandle)
* 取反图像
* 对字符图像取反，使字符极性适配 OCR 分类器
invert_image (ImageReduced, ImageInvert)
* 字符识别
* 对字符区域执行 OCR 识别
do_ocr_multi_class_mlp (SortedRegions, ImageInvert, OCRHandle, Class,
Confidence)

* 停止窗口刷新
* 关闭自动窗口刷新，提高显示控制稳定性
dev_update_window ('off')
* 设置字体
* 设置显示字体大小和样式
set_display_font (WindowHandle, 32, 'mono', 'true', 'false')
* 显示
dev_display (Image)
count_obj (SelectedRegions, Number)
* 循环显示识别出的字符
for index:=0 to Number-1 by 1
```

```
    disp_message (WindowHandle, Class[index], 'window', 12, 12+index*40,
'black', 'true')
endfor
stop()

* 划取汉字区域 ROI
* 以下注释代码为手动创建汉字模板的备选代码
* 生成汉字区域模板 ROI
*gen_rectangle1 (ModelRegion, 399.713, 486.753, 507.233, 539.66)
* 剪裁
* 提取汉字模板图像
*reduce_domain (Image, ModelRegion, TemplateImage)
* 创建模板
* 创建 NCC 模板，用于定位汉字区域
*create_ncc_model (TemplateImage, 'auto', rad(0), rad(360), 'auto',
'use_polarity', ModelID)

*
* Matching 01: ************************************************
* Matching 01: BEGIN of generated code for model initialization
* Matching 01: ************************************************
* Matching 01: Initialize acquisition
* 打开工业相机，用于实时采集图像并进行匹配识别
open_framegrabber ('GigEVision2', 0, 0, 0, 0, 0, 0, 'progressive', -1,
'default', -1, 'false', 'default', '0030532476d8_Basler_acA130060gm', 0,
-1, AcqHandle)
*
* Matching 01: Obtain the model image
* 读取参考图像，用汉字区域创建 NCC 模板
read_image (Image, 'E:/BCC/MVExp4/01.png')
*
* Matching 01: Draw a ROI interactively
* Matching 01: ATTENTION: you have to ensure that the window handle is
valid
* Matching 01: if you want to create a more complex region you have to
* Matching 01: insert the appropriate operations manually
gen_rectangle1 (ModelRegion, 399.713, 486.753, 507.233, 539.66)
*
* Matching 01: Reduce the model template
reduce_domain (Image, ModelRegion, TemplateImage)
*
* Matching 01: Create the correlation model
create_ncc_model (TemplateImage, 'auto', rad(0), rad(360), 'auto',
'use_polarity', ModelID)
*
* Matching 01: Set the reference position
* 设置模板参考点行坐标
RefRow := 454.5
```

```
* 设置模板参考点列坐标
RefColumn := 515
*
* Matching 01: Display the model region
dev_display (Image)
dev_set_color ('green')
dev_set_draw ('margin')
* 在参考位置生成十字轮廓显示模板中心
gen_cross_contour_xld (TransContours, RefRow, RefColumn, 20, 0.0)
dev_display (TransContours)
stop ()
*
* Matching 01: END of generated code for model initialization
* Matching 01:  * * * * * * * * * * * * * * * * * * * * * * * * *
* Matching 01: BEGIN of generated code for model application
*
* 进入实时匹配循环
while (true)
    *
    * Matching 01: Obtain the image
* 从相机抓取当前图像
    grab_image (Image, AcqHandle)
    *
    * Matching 01: Find the model
* 在当前图像中查找 NCC 模板，输出位置、角度和分数
    find_ncc_model (Image, ModelID, rad(0), rad(360), 0.8, 0, 0.5,
'true', 0, Row, Column, Angle, Score)
    *
    * Matching 01: Display the centers of the matches in the detected
positions
    dev_display (Image)
    for I := 0 to |Score| - 1 by 1
        * Matching 01: Display the center of the match
* 在匹配位置生成十字轮廓
        gen_cross_contour_xld (TransContours, Row[I], Column[I], 20,
Angle)
        dev_set_color ('green')
        dev_display (TransContours)
* 初始化用于图像对齐的二维变换矩阵
        hom_mat2d_identity (AlignmentHomMat2D)
* 将参考点平移到原点，为后续旋转做准备
        hom_mat2d_translate (AlignmentHomMat2D, -RefRow, -RefColumn,
AlignmentHomMat2D)
* 根据匹配角度构造旋转变换
        hom_mat2d_rotate (AlignmentHomMat2D, Angle[I], 0, 0,
AlignmentHomMat2D)
* 将模板坐标变换到当前匹配位置
```

```
        hom_mat2d_translate (AlignmentHomMat2D, Row[I], Column[I],
AlignmentHomMat2D)
        * Matching 01: Insert your code using the alignment here, e.g.
code generated by
        * Matching 01: the measure assistant with the code generation
option
        * Matching 01: 'Alignment Method' set to 'Affine Transformation'.
        stop ()
    endfor
    *
    * Matching 01: Code for rectification of the image
    * Matching 01: Calculate an inverse hom_mat2d for each of the
matching results
    for I := 0 to |Score| - 1 by 1
* 初始化图像矫正变换矩阵
        hom_mat2d_identity (RectificationHomMat2D)
* 根据匹配结果构造平移矫正变换
        hom_mat2d_translate (RectificationHomMat2D, RefRow-Row[I],
RefColumn-Column[I], RectificationHomMat2D)
* 根据匹配角度构造旋转矫正变换
        hom_mat2d_rotate (RectificationHomMat2D, -Angle[I], RefRow,
RefColumn, RectificationHomMat2D)
* 对当前图像进行仿射变换，得到矫正后的图像
        affine_trans_image (Image, RectifiedImage, RectificationHomMat2D,
'constant', 'false')
        *
        * Matching 01: Insert your code using the rectified image here
        rgb1_to_gray (RectifiedImage, GrayImage)
        * 获取ROI
* 生成英文和数字字符区域 ROI
        gen_rectangle1 (ROI_0, 394.593, 548.193, 508.94, 932.193)
        * 图片剪裁
* 将处理范围限制到字符 ROI
        reduce_domain (GrayImage, ROI_0, ImageReduced)
        * 阈值分割
* 按灰度阈值提取字符区域
        threshold (ImageReduced, Region, 128, 255)
        * 断开连通域
* 将字符区域拆分为独立连通域
        connection (Region, ConnectedRegions)
        * 特征选择
* 根据面积筛选有效字符连通域
        select_shape (ConnectedRegions, SelectedRegions, 'area', 'and',
600, 3000)
        * 连通域排序
* 按列方向对字符区域排序，保证识别顺序
        sort_region (SelectedRegions, SortedRegions, 'first_point',
'true', 'column')
```

```
        * 读分类器
* 读取 HALCON 内置 MLP OCR 分类器
        read_ocr_class_mlp('Industrial_0-9A-Z_NoRej.omc', OCRHandle)
        * 取反图像
* 对字符图像取反，使字符极性适配 OCR 分类器
        invert_image (ImageReduced, ImageInvert)
        * 字符识别
* 对字符区域执行 OCR 识别
        do_ocr_multi_class_mlp (SortedRegions, ImageInvert, OCRHandle,
Class, Confidence)
        * 显示
        dev_display (Image)
        count_obj (SelectedRegions, Number)
* 循环显示识别出的字符
        for index:=0 to Number-1 by 1
            disp_message (WindowHandle, Class[index], 'window', 12,
12+index*40, 'black', 'true')
        endfor
    endfor

    dev_display (TransContours)
    stop ()
endwhile
*
* Matching 01: ****************************************
* Matching 01: END of generated code for model application
* Matching 01: ****************************************
*
```